# Optimizing Block-Scale FP8 Mixture-of-Experts

Nikhil Deorkar

May 1, 2026

**Abstract**

This report describes the design and optimization of an FP8 block scale Mixture-of-Experts (MoE) kernel with DeepSeek V3 routing written in Triton. We also analyze the kernel's performance characteristics across test workloads and discuss the use of AI agent assistance during the development process. The kernel source is available at https://github.com/nikhilnd/mlsys26-flashinfer-moe.

# 1 Solution Overview

## 1.1 Tools & Languages

The solution is a PyTorch function that launches several Triton kernels. For simplicity, we refer to the solution as a single "kernel". Triton was selected for its ease of use and relatively low barrier to entry compared to other GPU programming tools. This enabled a greater iteration speed at the cost of sacrificing some granular control over the kernel.

## 1.2 Problem Description

The MoE calculation can be broken down into 5 steps: (1) DeepSeek V3 routing, (2) FP8 scaled GEMM 1, (3) SwiGLU activation, (4) FP8 scaled GEMM 2, and (5) output weighting. After routing, the PyTorch reference implementation[1] iterates over the local experts in a Python loop and performs the last 4 steps on the tokens selected by each expert. By contrast, the key design decision of our solution was to launch just a single kernel for GEMM 1 and SwiGLU and another single kernel for GEMM 2 and output accumulation. This design eliminates the need to synchronously iterate over experts and their assigned tokens. In the following section we describe the kernel design for each step of the problem in detail.

# 2 Kernel Design and Optimization

## 2.1 Routing

Our solution launches a custom routing Triton kernel to perform DeepSeek V3 auxiliary-loss-free routing using the input routing logits and bias. The kernel produces two $(T, K)$

[1] See reference implementation

output matrices representing the top-$K$ indices of experts for each token, and their corresponding routing weights (without bias). The launch grid is a 1D grid of $T$ tiles, representing a single tile for each token.

## 2.2 Expert Dispatch

To avoid launching separate GEMM 1/2 kernels for each expert, we compute a tile ordering such that all tokens in a single GEMM 1/2 tile are assigned to the same expert. Let $T'$ be the total number of (non-unique) tokens assigned to local experts. During expert dispatch, we compute the following:

- Sorted Tokens ($S$): a list of $T'$ token indices.
- Expert Start ($E$): a list of $E_{local} + 1$ non-decreasing integers such the tokens $S[E[i]], S[E[i] + 1], ..., S[E[i + 1] - 1]$ are assigned to expert $i$.

$S$ and $E$ are calculated with two helper Triton kernels. In the first pass, each tile scans a block of the flattened top-$K$ assignments and counts the tokens per local expert. These partial histograms are reduced and prefix-summed on the host to produce $E$, after which a second pass atomically scatters each token index into the correct output slot to produce $S$.

The total number of tokens assigned to expert $i$ is $E[i + 1] - E[i]$. For our GEMM kernels, we break this into $\lceil (E[i+1] - E[i])/B_M \rceil$ tiles where $B_M$ is a constant block size. This is similar to standard GEMM kernels, which break the $M$ dimension into $\lceil M/B_M \rceil$ tiles. In our case, $M = T'$ and the per-expert tile assignments make it trivial to launch just a single kernel for GEMM 1 and 2.

## 2.3 GEMM 1 + SwiGLU

Using the tile ordering calculated in the previous step, the GEMM 1 kernel looks very similar to a standard GEMM Triton kernel that computes $AW_1^\top$ where $A$ is $(T', H)$ and $W_1$ is $(2I, H)$. The launch grid is a 2D grid of shape $(\sum_{i=0}^{E_{local}-1} \lceil (E[i+1] - E[i])/B_M \rceil, \lceil I/B_{N_1} \rceil)$. We briefly describe some of the other specific optimizations made.

- **Token packing:** Each tile operates on tokens indicies from a subarray of the list $S$ computed during expert dispatch. Instead of randomly indexing into the original tensor of hidden states, we pack the tokens in a contiguous $(T', H)$ tensor where the $i$th row of the packed tensor is token $S[i]$. This allows for coalesced global accesses.
- **SwiGLU fusion:** The SwiGLU activation is fused into the GEMM 1 kernel. For tile $(i, j)$, we calculate both the columns at offset $j \cdot B_{N_1}$ and $j \cdot B_{N_1} + I$ and apply SwiGLU on the accumulators.
- **Block-scale FP8 output:** The output is quantized into a $(T', I)$ FP8 tensor of values and a $(\lceil I/B_N \rceil, T')$ FP32 tensor of scales using per-row max-abs scaling. This avoids materializing the full FP32 output in global memory and enables the use of FP8 tensor cores for GEMM 2.

### 2.4 GEMM 2 + Ouput Accumulation

Like GEMM 1, the GEMM 2 kernel is similar to a standard GEMM kernel computing $AW_2^\top$ where $A$ is $(T', I)$ and $W_2$ is $(H, I)$. The 2D launch grid has shape $(\sum_{i=0}^{E_{local}-1} \lceil (E[i+1]-E[i])/B_M \rceil, \lceil H/B_{N_2} \rceil)$. The GEMM 2 kernel also fuses the final output accumulation into the kernel epilogue by applying the weights calculated from the routing step and performing an atomic add on the output tensor by token index.

### 2.5 Other Optimizations

Both the GEMM 1 and GEMM 2 kernels use several other standard GEMM optimization techniques. These include: grouped tile launch ordering to increase L2 cache hit rate, FP8 tensor cores for matrix multiplications, and Triton autotuning to search for performant block sizes, warps per tile, and number of pipelined stages.

## 3 Performance Analysis

In this section, we describe some performance insights for our kernel. The raw performance for several selected workloads can be found in Table 1. Our kernel performs best compared to the FlashInfer baseline on workloads with a high number of tokens but underperforms on small workloads. This underperformance can likely be attributed to the overhead of the expert dispatch step. As shown in Table 2, expert dispatch dominates the runtime of small token workloads, while GEMM 1/2 are the largest components of large token workloads. Accordingly, the throughput percentage reported by Nsight Compute (NCU) for GEMM 1/2 is generally higher on large token workloads, as shown in Table 3.

| **Number of Tokens** | **Latency (ms)** | **Approx. Speedup vs. FlashInfer** |
|---|---|---|
| $n = 1$ | 0.636 | 0.10x |
| $n = 7$ | 0.668 | 0.14x |
| $n = 80$ | 0.968 | 0.28x |
| $n = 901$ | 1.046 | 0.67x |
| $n = 11984$ | 2.579 | 1.93x |
| $n = 14107$ | 3.440 | 1.85x |

Table 1: Raw Kernel Performance

| **Number of Tokens** | **Routing (ms)** | **Expert Dispatch (ms)** | **GEMM 1 (ms)** | **GEMM 2 (ms)** |
|---|---|---|---|---|
| $n = 7$ | 0.049 | 0.355 | 0.160 | 0.068 |
| $n = 14107$ | 0.082 | 0.214 | 1.372 | 1.360 |

Table 2: Performance Breakdown by Step

Additionally, we note some of the largest bottlenecks reported by NCU for the GEMM kernels on large token workloads:

| Number of Tokens | GEMM 1 | | GEMM 2 | |
|---|---|---|---|---|
| | Compute (%) | Memory (%) | Compute (%) | Memory (%) |
| $n = 7$ | 19.31 | 36.07 | 25.98 | 24.75 |
| $n = 14107$ | 46.61 | 35.39 | 44.31 | 42.06 |

Table 3: Memory and Throughput Percentage Breakdown

- Low occupancy: Both GEMM 1/2 use a high number of registers per thread (168 and 96, respectively), leading to a low theoretical occupancy of 3 and 5 warps per scheduler, respectively. GEMM 1 particularly requires heavy register usage to support the two accumulators for SwiGLU fusion. Kernel performance could potentially be improved by reducing register/shared memory per tile. However, this would also increase the total number of tiles, decrease Triton pipelining, or increase the risk of register spilling.
- Long scoreboard stalls: Warps on average spent 3 cycles waiting on L1TEX memory during GEMM 1 and an average of 5.2 cycles during GEMM 2. This indicates that the kernel's inner loop frequently stalls on loads from global memory.

# 4 Human vs. Agent Contributions

The following models were used for various tasks while developing our kernel:

- Gemini 3 Flash: Basic question answering for conceptual understanding.
- Claude Sonnet 4.6: Question answering, optimization idea generation, and basic coding tasks.
- Claude Opus 4.6: More advanced coding tasks, full agentic loop with Claude Code.

Sonnet 4.6 and Opus 4.6 were generally good at simple to moderately complex coding tasks, particularly translating existing PyTorch code into a custom Triton kernel. For example, Opus 4.6 wrote an optimized version of the expert dispatch utility function with only some light guidance. However, they struggled to fully generate correct GEMM kernels, and these were mostly written by hand.

When given an NCU output, the quality of optimizations suggested by Sonnet 4.6 and Opus 4.6 varied significantly. Although some were valuable, they often required human verification to determine whether the proposed optimization was worth pursuing.

Additionally, Opus 4.6 was given access to a B200 GPU and NCU to run an agent-driven optimization loop through Claude Code. However, during approximately 1 hour of running autonomously, it mainly fiddled with the kernel's blocks sizes and number of warps, achieving just a 2% speedup. The larger algorithmic changes it attempted were rolled back as they failed to improve performance. It is important to note, however, that the agent's baseline was already a significantly optimized version of the kernel.